

Multi-Tenancy

Keputusan Final

Sintesis dua rancangan — Opsi B dari deck tim, dan ADR-001 — menjadi satu rencana yang bisa langsung dieksekusi.



keputusan
yang sudah diambil

Delapan keputusan — dan dari mana asalnya

TITIK KEPUTUSAN	KEPUTUSAN	ASAL
Level hierarki	Tiga level: vendor → organization → perusahaan	ADR
Isolasi data	Satu database penuh per perusahaan	Keduanya
Hak akses (roles)	Di database pusat, pakai fitur teams Spatie	ADR
Perusahaan aktif	Di alamat URL, bukan di session	ADR
Identifikasi	Subdomain = organization, path = perusahaan	ADR
Implementasi	Package stancl/tenancy v3, bukan buatan sendiri	Deck
Session & queue	Disimpan di database pusat	Deck
Setup server	Wildcard DNS + SSL, grant MySQL berpola tenant_%	Deck

Semua keputusan di atas sudah final. Yang belum: tiga hal di slide 17.

Bentuk akhirnya

SATU CODEBASE

Satu repo, satu deploy — diperbaiki sekali, dirasakan semua perusahaan

DATABASE PUSAT

organizations · companies · users · roles & permissions · sessions · jobs · notifikasi

Menyimpan siapa boleh membuka perusahaan mana. Tidak menyimpan satu angka keuangan pun.

Kisantra

Pelanggan · 1 perusahaan

Kisantra

Mitra Group

Pelanggan · 10 perusahaan

PT Kinara

CV Naraya

CV Jitsugen

PT D

PT E

PT F

PT G

PT H

PT I

PT J

Setiap kotak merah = satu database terpisah. PT Kinara tidak punya jalan apa pun untuk melihat angka CV Naraya.

Perjalanan satu request

```
mitra-group.kisantra.com/c/pt-kinara/invoices
```

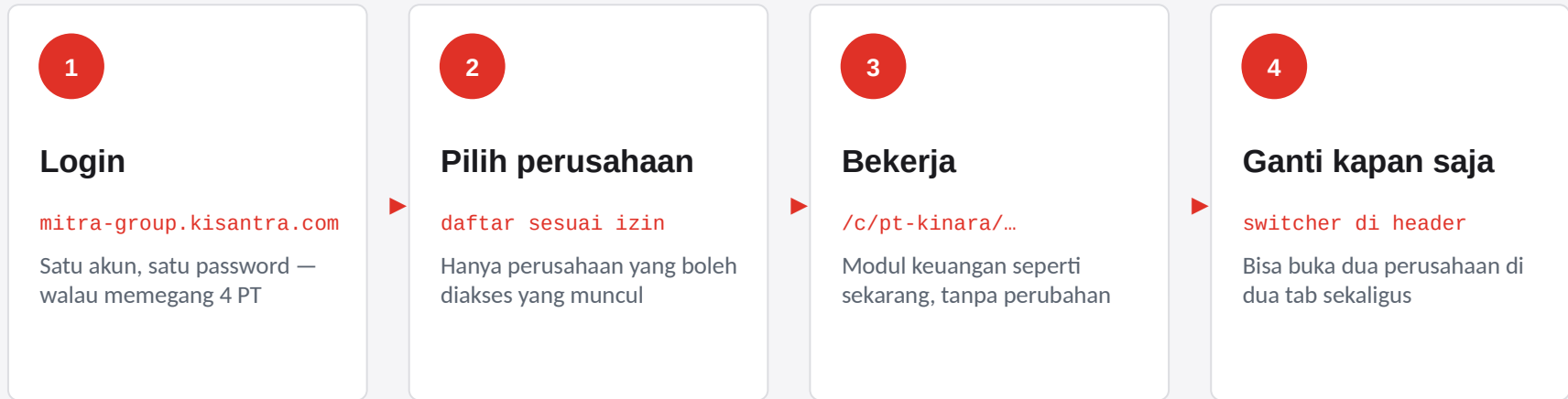
organization

perusahaan

- 1 **Baca subdomain** Ketemu organization Mitra Group di database pusat
- 2 **Cek login** User diautentikasi di database pusat
- 3 **Cek izin** Boleh buka pt-kinara? Kalau tidak → 403
- 4 **Tukar koneksi** Koneksi database diarahkan ke database PT Kinara
- 5 **Jalan normal** Invoice::all() otomatis membaca database yang benar

✓ Langkah 5 adalah kuncinya: seluruh modul keuangan tidak perlu diubah sama sekali.

Login sekali, pegang beberapa perusahaan



Kenapa satu akun, bukan sepuluh

- Akuntan Mitra Group yang memegang 4 PT cukup punya satu akun.
- Saat ia keluar, aksesnya dimatikan sekali — bukan empat kali di empat tempat.
- Perannya bisa berbeda per perusahaan: manajer keuangan di PT Kinara, staff di CV Naraya.

Kenapa perlu level organization

Deck tim hanya punya dua level: pusat → perusahaan. User menempel langsung ke perusahaan.

Untuk 7 perusahaan satu grup internal itu cukup. Untuk kondisi sekarang, tiga hal jadi tidak punya tempat.

1

Kuota

Tidak ada tempat menyimpan "Mitra Group berkuota 10 perusahaan". Kuota menempel ke pelanggan — dan pelanggan tidak ada di model.

2

Batas admin

Kalau admin Mitra Group boleh menambah perusahaan sendiri, ia harus jadi admin dari sesuatu. Tanpa organization, tidak ada objek yang bisa jadi batas.

3

Namespace

Subdomain jadi datar. Dua pelanggan berbeda dengan anak perusahaan bernama mirip akan bertabrakan slug-nya secara global.

✓ Biayanya kecil: satu tabel organizations, satu kolom organization_id di users. Tanpa itu, penagihan langganan dan pembuatan perusahaan mandiri tidak punya pondasi.

Hak akses disimpan di pusat, bukan per perusahaan

Deck tim: tabel Spatie per perusahaan

- Lebih sederhana diimplementasikan.
- Tiap perusahaan bebas punya peran sendiri.
- Tidak perlu fitur teams Spatie.

Konsekuensi: hak akses tidak bisa dibaca tanpa berpindah koneksi lebih dulu.

Pilihan kita: di pusat, pakai teams

- Satu tempat mengelola izin seluruh grup.
- Peran per perusahaan tetap didukung.
- Satu cache izin — tidak ada risiko tertukar.

Migration Spatie di project sudah memuat cabang teams — mengaktifkan, bukan menambal.

Dua fitur yang putus kalau izin disimpan per perusahaan

Bell notifikasi lintas perusahaan

Menampilkan peringatan PT Kinara dan CV Naraya dalam satu daftar butuh tahu izin di keduanya — tanpa berpindah konteks.

Halaman kelola user terpadu

Pertanyaan "Budi punya akses ke apa saja" jadi sepuluh kali koneksi database, bukan satu query.

! Perlu dibereskan lebih dulu: deck tim menyebut role ada di pivot company_user (slide 14) sekaligus di tabel Spatie per database (slide 13 & 30). Dua sumber kebenaran untuk hal yang sama — harus dipilih satu sebelum coding.

Perusahaan aktif ada di alamat, bukan di session

Deck tim memilih session dan menyebut trade-off-nya "satu browser satu perusahaan aktif". Untuk aplikasi keuangan, konsekuensinya lebih berat dari itu.

Kalau disimpan di session

- Tab 1 buka PT Kinara
- Tab 2 buka CV Naraya → session berubah
- Tab 1 **edit invoice — masuk ke CV Naraya**

Tab 1 tidak pernah tahu konteksnya sudah pindah.

Kalau ada di alamat

- Dua tab, dua perusahaan — aman berdampingan.
- Link bisa dikirim ke rekan tanpa salah perusahaan.
- Tombol back berperilaku benar.
- Bookmark menunjuk perusahaan yang tepat.

! Ini bukan gangguan kenyamanan — ini jalur salah-posting. Akuntan yang memegang beberapa PT hampir pasti membandingkan berdampingan, dan tidak ada peringatan apa pun saat konteks tab lama sudah berubah.

✓ Sekaligus menyelesaikan kontradiksi di deck tim: slide 16–17 merekomendasikan tanpa subdomain, tapi slide 13, 21, 22, dan 23 mengimplementasikan subdomain per perusahaan. Pilihan kita memakai keduanya pada level berbeda.

Pakai package, jangan tulis sendiri

ADR-001 mendeskripsikan TenantManager buatan sendiri. Deck tim menyebut stanc/tenancy v3 — dan itu pilihan yang lebih baik. Empat hal yang sudah ditangani package, yang kalau ditulis sendiri harus dibenarkan satu per satu:

Queue

Tenant id otomatis disisipkan ke payload job — worker tahu perusahaan mana

Cache

Cache di-namespace per perusahaan — termasuk cache izin Spatie

Storage

Folder lampiran otomatis terpisah per perusahaan

Provisioning

Event TenantCreated: buat database, migrate, seed, siapkan folder

Penyesuaian dari contoh di deck tim

- Deck memakai InitializeTenancyByDomain (perusahaan = subdomain). Kita pakai identifikasi lewat path, karena perusahaan ada di URL.
- Subdomain tetap dipakai — tapi untuk mengenali organization, ditangani middleware kita sendiri.

! Wajib dilakukan lebih dulu: baca dokumentasi resmi stanc/tenancy dan buktikan identifikasi lewat path bekerja seperti yang kita asumsikan. Ini salah satu dari tiga hal yang belum diverifikasi.

Bagian terbesar sistem tidak tersentuh

TIDAK BERUBAH

17 modul keuangan

- Invoice, item, pembayaran
- Arus kas, rekening bank, transfer
- Kategori transaksi
- Laporan Laba Rugi
- Utang, piutang
- Reimbursement, permintaan dana
- Invoice berulang
- Penomoran dokumen — tetapurut per perusahaan
- Pengaturan PKP, NPWP, tarif PPN per perusahaan
- Semua template dan cetakan PDF

PERLU DIKERJAKAN

Area login & hak akses

- Daftar user & password pindah ke pusat
- Hak akses jadi per perusahaan
- Halaman kelola pengguna & peran
- Alamat web mendapat nama perusahaan
- Halaman baru: pilih perusahaan
- Tombol ganti perusahaan di header
- Lampiran dipisah per perusahaan
- Notifikasi & feedback pindah ke pusat
- Foreign key ke users dilepas (lintas database)

Bandingkan dengan Opsi A: kolom baru di ±20 tabel, penomoran ditulis ulang, dan setiap query agregat harus diaudit.

Setup server — sekali saja

DNS, nginx, SSL

```
*.kisantra.com    A    203.0.113.10

server_name      *.kisantra.com;
root /var/www/finance/public;

certbot -d "/*.kisantra.com"
```

Hak akses MySQL

```
GRANT ALL ON `kisantra_core`.* TO app;
GRANT ALL ON `tenant\_%\`.*      TO app;

DB_DATABASE=kisantra_core
TENANCY_DB_PREFIX=tenant_
```

✓ Pola tenant_% itu detail keamanan yang bagus dari deck tim: user aplikasi hanya bisa menyentuh database berprefix tenant_, tidak bisa mengintip database lain di server yang sama.

! Satu pengetatan dari deck: GRANT CREATE ON *.* terlalu longgar — itu mengizinkan pembuatan database apa pun. Pakai kredensial terpisah khusus provisioning, jangan diberikan ke user aplikasi yang melayani request.

Subdomain baru untuk pelanggan baru langsung hidup tanpa menyentuh server — wildcard DNS dan SSL sudah menangkap semuanya.

Menambah perusahaan baru

Menambah klien di modul Clients cuma menyisipkan satu baris. Menambah perusahaan berarti membuat database baru — jadi bentuknya proses latar belakang berstatus, bukan form yang langsung selesai.

1

Isi form

nama, singkatan, kuota dicek

2

Buat database

tenant_kinara

3

Jalankan migration

55 file

4

Isi data awal

profil, kategori master

5

Siap dipakai

status → active

Dua hal yang dikunci saat pembuatan

- Kode URL permanen — ia ada di alamat web dan di path lampiran. Mengubahnya nanti memutus semua bukti bayar.
- Singkatan dokumen wajib diisi manual.

Kenapa singkatan tidak boleh otomatis

PT. Kinara → **K**

CV. Naraya → **N**

CV. Jitsugen → **J**

Nomor invoice jadi 001/INV/K-ABC/... — ambigu di mata klien.

! Kalau gagal di tengah jalan, ada perusahaan setengah jadi. Tabel companies wajib punya status provisioning / active / failed, plus tombol coba-ulang dan pembersihan database sampah.

Deploy ke banyak database

Setiap rilis

```
php artisan migrate --force  
php artisan tenants:migrate --force  
php artisan queue:restart
```

Backup malam

Satu dump per database. Restore satu perusahaan tanpa menyentuh yang lain — dan data satu klien bisa diserahkan utuh ke auditornya.

Tiga pengaman yang wajib ada

1

Expand-contract

Kode versi baru harus jalan di skema lama dan baru, supaya migrasi dan deploy tidak wajib serentak

2

Canary

Migrasi satu perusahaan dulu, verifikasi, baru lanjut ke sisanya

3

Laporan per perusahaan

Gagal di perusahaan ke-7? Harus jelas enam mana yang sudah dan tiga mana yang belum

! Konsekuensi yang harus diterima sadar: sekarang deploy buruk merusak satu perusahaan. Setelah ini, merusak semuanya.

Tujuh tahap eksekusi

0	Beresi dulu	Satukan codebase, perbaiki bug yang diketahui, pindahkan CI ke MySQL	WAJIB DULU
1	Database pusat	organizations, companies, users, tabel Spatie. Tarik user dari tiap deployment	
2	Routing & pemilih	Middleware, segmen URL, halaman pilih perusahaan, switcher	
3	Efek samping	Folder lampiran, notifikasi, queue & scheduler per perusahaan	
4	Alat operasional	Provisioning otomatis, backup per perusahaan, canary, monitoring	
5	Hapus hardcode	Tarif pajak jadi pengaturan, template invoice ke penyusun template	NANTI
6	Laporan gabungan	Kalau diminta. Prasyaratnya dikerjakan sejak Tahap 4	NANTI

Setelah Tahap 2, Mitra Group sudah bisa dipakai. Tahap 5 dan 6 baru mendesak kalau memang dibutuhkan.

Empat hal yang harus beres lebih dulu

Satukan codebase

1

Pastikan semua deployment berasal dari kode yang sama. Kalau ada yang sudah dimodifikasi di luar file template Blade, normalisasi jadi konfigurasi dulu.

Tanpa ini, satu-codebase mustahil

Perbaiki bug yang diketahui

2

Lookup kolom code di LoanController dan ReceivableController — kolomnya sudah dihapus migration Februari, jadi create loan dan approve piutang gagal 500 di MySQL.

Perbaiki sekali, semua ikut

Pindahkan CI ke MySQL

3

Test suite masih SQLite in-memory, dan itu sudah terbukti menyembunyikan bug di atas.

Jaring pengaman sebelum menyentuh infrastruktur

Samakan versi migrasi

4

Yang menentukan besar Tahap 1 bukan jumlah perusahaan, tapi selisih versi antar deployment.

Belum diukur

Tidak ada di deck tim — deck ditulis seolah memulai dari nol. Kenyataannya sudah ada beberapa deployment berjalan.

Enam risiko dan penangkalnya

RISIKO	PENANGKAL	ASAL
Deploy buruk kena semua perusahaan	Perusahaan uji coba + canary + expand-contract + rollback teruji	ADR
Migrasi gagal di tengah daftar	Runner idempoten, status per perusahaan, alert saat gagal	Deck
Saling memperlambat di level server	Export PDF besar dipindah ke proses latar belakang	ADR
Database pusat jadi titik vital	Backup lebih ketat + monitoring khusus, beda dari database biasa	ADR
Test suite melambat	Database tenant template + pisahkan test pusat dan perusahaan	Deck
Ketergantungan pada package	Pakai standar de facto; baca dokumentasi resmi sebelum implementasi	Deck

Risiko nomor 3 punya angka konkret: layanan export arus kas menaikkan batas memori ke 1 GB selama 5 menit. Sepuluh perusahaan berbagi satu server aplikasi.

Tiga dari ADR, tiga dari deck tim — tidak ada satu dokumen yang menangkap semuanya.

Tiga hal yang belum diverifikasi

1

Identifikasi lewat path di stancI/tenancy

Teknis

Kita berasumsi package mendukung pengenalan perusahaan dari segmen URL, bukan hanya dari subdomain. Harus dibuktikan di dokumentasi resmi sebelum arsitektur dikunci.

2

Di mana role sebenarnya disimpan

Teknis

Deck tim menyebut dua tempat sekaligus: pivot company_user dan tabel Spatie per database. Harus dipilih satu.

3

Peran disamakan grup atau bebas per perusahaan

Produk

Kalau "Manajer Keuangan" didefinisikan sekali untuk Mitra Group, mengubah izinnya mengubah untuk kesepuluh PT. Lebih rapi, tapi PT tidak bisa punya aturan sendiri.



Dua yang pertama menghalangi coding. Yang ketiga bisa diputuskan paralel, tapi lebih murah sekarang daripada setelah data izin terisi.

Yang bisa dimulai minggu ini

1

Ukur version drift

Bandingkan status migration seluruh deployment yang berjalan. Ini yang menentukan besar pekerjaan Tahap 1 — dan angkanya belum ada.

Setengah hari

2

Baca dokumentasi stanci/tenancy

Khususnya identifikasi lewat path dan tenancy untuk queue serta cache. Buktikan asumsi di slide 9.

Satu hari

3

Perbaiki bug kolom code

Ganti mekanisme pencarian kategori sistem di modul Utang dan Piutang. Berdiri sendiri, tidak menunggu keputusan apa pun.

Satu hari

4

Pindahkan CI ke MySQL

Jalankan test suite di MySQL, bukan SQLite in-memory. Kemungkinan akan memunculkan kegagalan yang selama ini tersembunyi.

Satu-dua hari

Keempatnya adalah Tahap 0. Tidak satu pun menunggu keputusan arsitektur, dan semuanya berguna walau arahnya nanti berubah.

"Kompleksitas multi-tenant tidak hilang — ia hanya berpindah: Opsi A menaruhnya di kode, Opsi B menaruhnya di infrastruktur."

— deck tim, slide 31

Opsi A • kompleksitas di kode

gagal senyap — angka salah, tanpa error

Opsi B • kompleksitas di infrastruktur

gagal keras — langsung kelihatan

Untuk aplikasi keuangan, mode kegagalan yang senyap adalah yang paling mahal. Itu alasan kita memilih infrastruktur.

Langkah berikutnya: jawab tiga pertanyaan di slide 17, mulai empat pekerjaan di slide 18.